BuildAtlas — MERN Full-Stack Developer Project Knowledge Platform

1. Project Overview

Build a production-grade full-stack web platform called **BuildAtlas**.

Tagline

Explore how real software is designed, built, deployed, and learned from.

BuildAtlas is a developer-focused project knowledge and discovery platform where developers document and explore the complete engineering story behind software projects.

GitHub primarily shows the code.

LinkedIn primarily shows the developer.

BuildAtlas shows **how the software was actually designed, built, deployed, and improved**.

A developer should be able to create a project page that documents:

- What the project does
- Why it was built
- Features
- Technology stack
- System architecture
- Database design
- API design
- Engineering decisions
- Problems encountered
- Failed approaches
- Solutions
- Trade-offs
- Performance considerations
- Security considerations
- Deployment architecture
- Development timeline
- Lessons learned
- Future improvements

The goal is to create a platform where another developer can understand not only **what a project contains**, but **why it was built the way it was**.

---

2. Core Problem

Developers currently use multiple platforms to understand software projects.

GitHub provides:

- Source code
- README files
- Issues
- Pull requests
- Commits

LinkedIn provides:

- Developer profiles
- Career information

Blogs provide:

- Technical explanations

YouTube provides:

- Development walkthroughs

However, there is no unified platform focused on documenting the complete engineering journey of a project.

For example, someone who wants to build:

- CRM
- Expense Tracker
- AI Resume Analyzer
- Trading Platform
- E-commerce application
- SaaS platform
- AI agent
- Developer tool

can find GitHub repositories but often cannot quickly answer:

- Why was MongoDB chosen?
- Why was Redis introduced?
- Why was a particular architecture selected?
- What were the alternatives?

- What problems appeared during development?
- What failed?
- How was authentication implemented?
- How was the application deployed?
- What performance issues occurred?
- What would the developer change if rebuilding it?
- What lessons were learned?

BuildAtlas solves this by turning a software project into a structured engineering case study.

---

3. Product Philosophy

BuildAtlas must NOT feel like:

- A GitHub clone
- A basic CRUD application
- A generic portfolio builder
- A simple blogging platform
- A generic social network

It should feel like a combination of:

- Developer portfolio
- Project discovery platform
- Architecture documentation
- Engineering case study
- Technical knowledge base
- Developer community

The primary objective is:

Make software projects understandable.

---

4. Target Users

Developers

Document projects and explain engineering decisions.

Students

Show technical depth beyond a project title and GitHub link.

Open-source Developers

Document architecture and development decisions beyond README files.

Recruiters

Evaluate actual engineering ability and project depth.

Developers Learning from Projects

Search for real projects using specific technologies and architectures.

Example:

A developer searching for:

React + Node.js + MongoDB SaaS application

should be able to find relevant projects and study how those systems were built.

---

5. Core Technology Stack

The application must be built using the **MERN stack with TypeScript**.

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- React Router
- TanStack Query
- React Hook Form
- Zod
- React Flow
- Recharts

Backend

- Node.js
- Express.js
- TypeScript

Database

- MongoDB
- Mongoose

Caching / Background Jobs

- Redis
- BullMQ

Authentication

- JWT
- Refresh token architecture
- bcrypt or Argon2

File Storage

- AWS S3 or S3-compatible storage

External Integration

- GitHub REST API
- GitHub OAuth

Testing

- Vitest
- Supertest
- React Testing Library

API Documentation

- Swagger / OpenAPI

DevOps

- Docker
- Docker Compose
- GitHub Actions
- Nginx

Cloud

- AWS

Monitoring

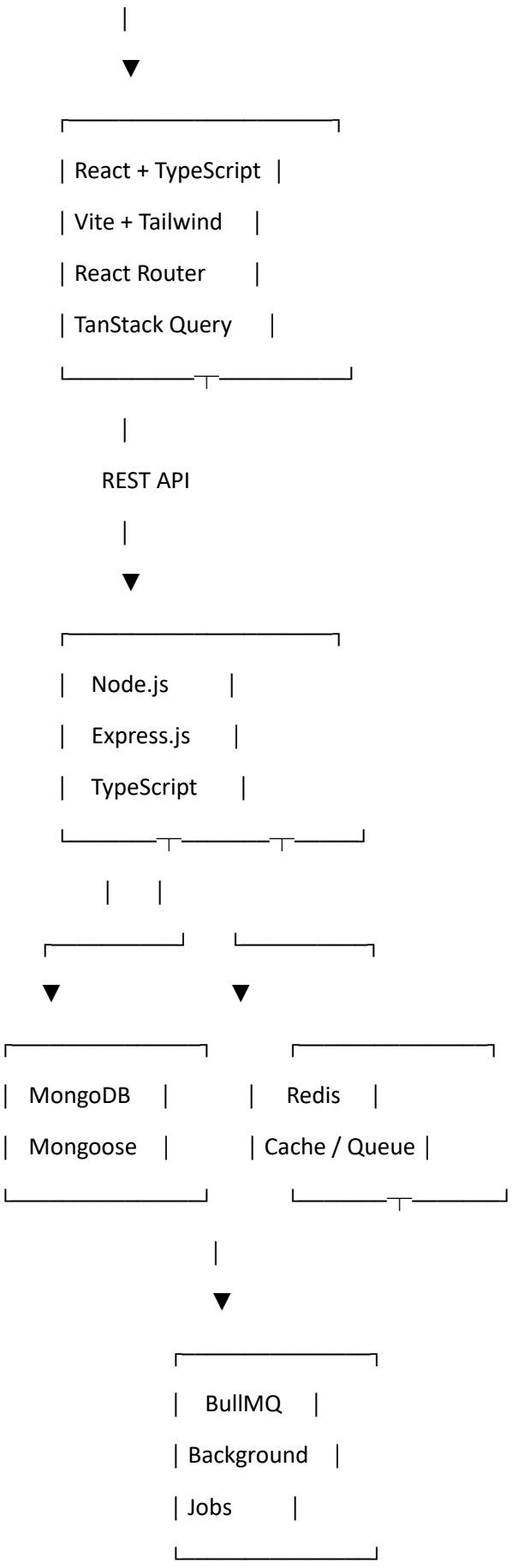
- Sentry

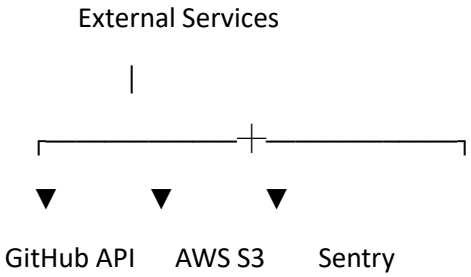
---

6. MERN Architecture

The application should follow a clean full-stack architecture.

USER


React + TypeScript
Vite + Tailwind
React Router
TanStack Query
REST API
Node.js
Express.js
TypeScript
MongoDB
Mongoose
Redis
Cache / Queue
BullMQ
Background
Jobs



---

7. Frontend Architecture

Use a feature-oriented React architecture.

Recommended structure:

frontend/

src/

app/

components/

layouts/

pages/

features/

auth/

projects/

profiles/

discovery/

architecture/

database/

api-docs/

decisions/

problems/

timeline/

deployment/

comments/

bookmarks/

notifications/

analytics/

hooks/

services/

lib/

routes/

types/

utils/

Avoid putting application logic into large components.

Create reusable UI components.

Use TypeScript throughout the frontend.

---

8. Backend Architecture

Use a modular Express.js architecture.

Recommended structure:

backend/

src/

server.ts

app.ts

config/

env.ts

database.ts

routes/

auth.routes.ts

users.routes.ts

projects.routes.ts

technologies.routes.ts

architecture.routes.ts

database.routes.ts

apiDocs.routes.ts

decisions.routes.ts

problems.routes.ts

timeline.routes.ts

deployment.routes.ts

comments.routes.ts

likes.routes.ts

bookmarks.routes.ts

follows.routes.ts

notifications.routes.ts

analytics.routes.ts

github.routes.ts

admin.routes.ts

controllers/

services/

repositories/

models/

schemas/

middleware/

auth.middleware.ts

error.middleware.ts

rateLimit.middleware.ts

upload.middleware.ts

utils/

jobs/

Use this request flow:

Route

↓

Middleware

↓

Controller

↓

Service

↓

Repository / Model

↓

MongoDB

Business logic should not be placed directly inside route files.

---

9. Authentication

Implement secure authentication.

Features:

- Registration
- Login
- Logout
- JWT access tokens
- Refresh tokens
- Password hashing
- Password reset architecture
- Email verification architecture
- Protected routes
- Role-based authorization

User roles:

USER

ADMIN

Only project owners can edit their projects.

Admins can moderate the platform.

Use secure HTTP-only cookies where appropriate for refresh tokens.

Never store sensitive credentials in frontend local storage.

---

10. User Profiles

Each user should have a public developer profile.

Profile fields:

- Name
- Username
- Profile picture
- Bio
- Location
- Website
- GitHub
- LinkedIn
- Skills
- Technologies
- Projects
- Followers
- Following

Profile sections:

Overview

Developer introduction.

Skills

Technology list.

Projects

Published projects.

Activity

Recent activity.

Social

Followers and following.

Public profile URL:

/u/:username

---

11. Project Creation

Users should be able to create projects through a multi-step project builder.

Project fields:

- Name
- Slug
- Short description
- Full description
- Category
- Project type
- Difficulty
- Status
- Start date
- End date
- Repository URL
- Live URL
- Demo URL
- Documentation URL
- License
- Cover image
- Project logo
- Visibility

Project types:

- Web Application
- SaaS
- Mobile Application

- AI/ML
- Developer Tool
- Open Source
- Desktop Application
- Game
- API
- Data Platform
- Cybersecurity
- Other

Project status:

- Planning
- Active Development
- Production
- Maintained
- Archived

Visibility:

- Draft
- Public
- Private

---

12. Technology Stack

Every project should have a structured technology stack.

Technology categories:

Frontend

Examples:

- React
- Next.js
- Vue
- Angular

Backend

Examples:

- Node.js
- Express
- NestJS
- Django
- Spring Boot

Database

Examples:

- MongoDB
- PostgreSQL
- MySQL
- Redis

AI / ML

Examples:

- OpenAI
- Anthropic
- Hugging Face
- PyTorch

Infrastructure

Examples:

- AWS
- Docker
- Kubernetes
- Cloudflare

Tools

Examples:

- Git
- GitHub
- Postman
- Terraform

Technologies should be searchable and filterable.

---

13. Architecture Documentation

Architecture is one of the most important parts of the platform.

Each project should have an interactive architecture section.

Allow users to create:

- Frontend nodes
- Backend nodes
- Database nodes
- Cache nodes
- Storage nodes
- External API nodes
- Queue nodes
- Worker nodes
- Infrastructure nodes

Use **React Flow** for the architecture editor.

Users should be able to:

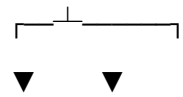
- Drag nodes
- Create connections
- Add labels
- Edit nodes
- Delete nodes
- Group nodes
- Zoom
- Pan
- Save diagrams
- Preview diagrams

Store diagrams as structured JSON.

Example:

React Frontend

Express API



MongoDB Redis

BullMQ

Do not store architecture only as a static image.

---

14. Engineering Decisions

Create a dedicated:

Architecture & Engineering Decisions

section.

Each decision should contain:

- Title
- Problem
- Context
- Options considered
- Selected solution
- Reason
- Trade-offs
- Consequences
- Status
- Date

Example:

Decision:

MongoDB vs PostgreSQL

Problem:

The project required flexible project documentation structures.

Options:

MongoDB

PostgreSQL

Decision:

MongoDB

Reason:

Project documentation contains highly variable nested structures.

Trade-offs:

Less rigid schema compared to relational databases.

This should resemble lightweight Architecture Decision Records.

---

15. Problems & Solutions

Developers should document real development problems.

Each entry:

- Problem title
- Description
- Symptoms
- Root cause
- Investigation
- Failed approaches
- Final solution
- Result
- Lessons learned

Example:

Problem:

Project discovery became slow.

Investigation:

Large MongoDB queries were scanning unnecessary documents.

Solution:

Added indexes and optimized query filters.

Result:

Improved project discovery response time.

---

16. Development Timeline

Create a project timeline.

Timeline events:

- Project started
- Architecture designed
- MVP completed
- Authentication added
- Database redesigned
- Major feature released
- Deployment
- Major bug fixed
- Version released

Each event:

- Date
- Title
- Description
- Optional image
- Optional GitHub reference

Display the timeline visually.

---

17. API Documentation

Allow developers to document APIs.

Each API endpoint should contain:

- HTTP method

- Endpoint
- Description
- Authentication requirement
- Parameters
- Request body
- Response body
- Status codes
- Example request
- Example response

Example:

POST /api/v1/projects

Request:

```
{
  "name": "BuildAtlas"
}
```

Response:

```
{
  "id": "...",
  "name": "BuildAtlas"
}
```

Provide a clean API explorer UI.

---

18. Database Documentation

Allow developers to document MongoDB data models.

Document:

- Collections
- Fields
- Data types
- Required fields

- References
- Embedded documents
- Indexes
- Validation rules

Provide a visual database relationship diagram.

Example:

User

│

├── Projects

│

└── Comments

Project

│

├── Technologies

├── Architecture

├── Decisions

├── Problems

└── Timeline

Because MongoDB is document-oriented, clearly distinguish:

- Embedded documents
- Referenced documents

Allow users to attach schema/code snippets.

---

19. Deployment Documentation

Projects should have a deployment section.

Document:

- Cloud provider
- Frontend hosting
- Backend hosting
- Database hosting

- Object storage
- CDN
- CI/CD
- Docker
- Domain
- Environment configuration

Example:

React

↓

Cloud hosting

Express API

↓

AWS EC2 / ECS

MongoDB

↓

MongoDB Atlas

Images

↓

AWS S3

CI/CD

↓

GitHub Actions

---

20. Lessons Learned

Create:

What I Learned

Developers can document:

- Technical lessons
- Architecture lessons
- Performance lessons
- Security lessons
- Product lessons
- Mistakes
- Development lessons

Also provide:

If I Rebuilt This Project

Developers explain what they would change.

This is one of the platform's main differentiators.

---

21. Project Discovery

Create a dedicated discovery page.

Sections:

- Trending
- Recently Published
- Recently Updated
- Popular
- Most Viewed
- Most Bookmarked
- Recommended

Project cards display:

- Project name
- Cover image
- Description
- Author
- Technologies
- Category
- Views
- Likes

- Bookmarks

---

22. Search

Implement project search.

Search across:

- Project name
- Description
- Technologies
- Category
- Author
- Keywords
- Architecture metadata

Examples:

React MongoDB SaaS

Node.js AI project

Redis microservices

Cybersecurity

Computer Vision

Implement:

- Text search
- Filters
- Sorting
- Pagination

Use MongoDB indexes appropriately.

Design the search layer so semantic search can be added later.

---

23. Filtering

Provide filters for:

Technology

MongoDB

React

Node.js

Python

Java

C++

etc.

Category

- AI
- SaaS
- Web
- Mobile
- Cybersecurity
- Developer Tools

Architecture

- Monolith
- Microservices
- Serverless
- Event-driven

Difficulty

- Beginner
- Intermediate
- Advanced

Status

- Production
- Active
- Archived

---

24. Bookmarks

Users can bookmark projects.

Features:

- Bookmark
- Remove bookmark

- Saved projects
- Bookmark collections

Potential collections:

AI Projects

SaaS Ideas

Architecture References

Projects to Study

---

25. Likes

Users can like projects.

Implement:

- Like
- Unlike
- Like count

Prevent duplicate likes.

Use appropriate MongoDB indexes and unique constraints/logic.

---

26. Comments

Users can comment on projects.

Features:

- Add comment
- Edit own comment
- Delete own comment
- Reply
- Like comments
- Report comments

Support threaded replies.

---

27. Following

Users can follow developers.

Features:

- Follow
- Unfollow
- Followers
- Following
- Activity feed

The activity feed should display projects published or updated by followed users.

---

28. Notifications

Implement notifications for:

- New follower
- Project like
- Bookmark
- Comment
- Comment reply

Use Redis/BullMQ for asynchronous notification processing where appropriate.

---

29. Project Analytics

Project owners should have access to analytics.

Track:

- Total views
- Unique visitors
- Likes
- Bookmarks
- Comments
- Followers gained
- Views over time
- Popular project sections

Dashboard should include charts using Recharts.

Filters:

7 days

30 days

90 days

All time

Do not fabricate analytics.

Only display actual tracked data.

---

30. GitHub Integration

Allow users to connect GitHub repositories.

Use GitHub OAuth/API.

Retrieve:

- Repository name
- Description
- Stars
- Forks
- Primary language
- Topics
- License
- Recent commits
- Contributors
- Repository URL

GitHub should supplement BuildAtlas.

BuildAtlas remains the structured engineering documentation layer.

---

31. GitHub Project Import

Provide:

Import from GitHub

User selects a repository.

Prepopulate:

- Project name
- Description
- Languages
- Topics

- License
- Repository URL
- README

The user must manually complete:

- Architecture
- Engineering decisions
- Problems
- Deployment
- Lessons

Do not invent engineering information.

---

32. Project Page

The project page is the primary product experience.

Structure:

Header

- Project name
- Short description
- Author
- Technologies
- GitHub
- Live Demo
- Like
- Bookmark

Overview

What the project does.

Why I Built It

Motivation.

Features

Main functionality.

Tech Stack

Technology categories.

Architecture

Interactive architecture.

Database

MongoDB schema/model documentation.

APIs

API documentation.

Engineering Decisions

Architecture decisions.

Problems & Solutions

Development challenges.

Timeline

Development history.

Deployment

Infrastructure.

Lessons Learned

Engineering lessons.

Future Improvements

Roadmap.

Community

Comments and interactions.

---

33. Project Editor

Create a professional project dashboard.

Sidebar:

Overview

Features

Tech Stack

Architecture

Database

APIs

Decisions

Problems

Timeline

Deployment

Lessons

Settings

Features:

- Save draft
- Autosave
- Preview
- Publish
- Unpublish
- Last saved indicator
- Last updated indicator

Use optimistic updates only where safe.

---

34. Rich Text Editor

Implement a professional editor.

Support:

- Headings
- Bold
- Italic
- Lists
- Code blocks
- Inline code
- Links
- Images
- Tables
- Quotes

Code blocks must support syntax highlighting.

Store structured content rather than unsafe raw HTML.

Sanitize rendered content.

---

35. File & Asset Management

Users can upload:

- Project screenshots
- Architecture images
- Logos
- Diagrams
- Demo images

Store files in:

AWS S3

Do not store large files directly inside MongoDB.

Create a storage abstraction so local development can use local storage while production uses S3.

Validate:

- MIME type
- File extension
- File size
- Upload authorization

---

36. Admin Dashboard

Create an admin dashboard.

Admin capabilities:

- View users
- Search users
- View projects
- Search projects
- Suspend users
- Hide projects
- Delete comments
- Review reports
- View platform analytics

Metrics:

- Total users
- Active users
- Total projects
- Published projects
- Comments
- Likes
- Views

---

37. Reporting System

Users can report:

- Projects
- Comments
- Users

Reasons:

- Spam
- Abuse
- Copyright
- Malicious content
- Misleading content
- Other

Admins can:

- Review
- Resolve
- Reject
- Take moderation action

---

38. SEO

Public project pages must be SEO-friendly.

Implement:

- Dynamic titles
- Meta descriptions

- Open Graph metadata
- Canonical URLs
- Sitemap
- robots.txt
- Semantic HTML

URLs:

/projects/:slug

/u/:username

---

39. Responsive Design

Application must work on:

- Desktop
- Laptop
- Tablet
- Mobile

Architecture editor can prioritize desktop, but public project pages must be highly responsive.

---

40. UI/UX

Design should be professional and developer-focused.

Avoid:

- Excessive gradients
- Generic AI aesthetics
- Excessive glassmorphism
- Unnecessary animations
- Fake metrics
- Excessive rounded cards
- Cluttered interfaces

Prioritize:

- Typography
- Information hierarchy

- Spacing
- Accessibility
- Clear navigation
- Good information density
- Consistent components

The product should feel like a serious developer platform.

---

41. MongoDB Data Architecture

Use MongoDB with Mongoose.

Important collections:

users

projects

technologies

projectTechnologies

architectureDiagrams

databaseSchemas

apiEndpoints

engineeringDecisions

problems

timelineEvents

deployments

lessons

projectViews

likes

bookmarks

comments

follows

notifications

reports

githubRepositories

projectAssets

Use:

- ObjectId references
- Embedded documents where appropriate
- Compound indexes
- Unique indexes
- TTL indexes where appropriate
- Schema validation

Avoid excessive document embedding for high-growth collections such as:

- Comments
- Views
- Notifications
- Likes

These should generally be separate collections.

---

42. API Design

Build REST APIs under:

/api/v1

Examples:

POST /api/v1/auth/register

POST /api/v1/auth/login

POST /api/v1/auth/logout

POST /api/v1/auth/refresh

GET /api/v1/users/:username

PATCH /api/v1/users/me

GET /api/v1/projects

POST /api/v1/projects

GET /api/v1/projects/:slug

PATCH /api/v1/projects/:id

DELETE /api/v1/projects/:id

POST /api/v1/projects/:id/publish

POST /api/v1/projects/:id/like

DELETE /api/v1/projects/:id/like

POST /api/v1/projects/:id/bookmark

DELETE /api/v1/projects/:id/bookmark

GET /api/v1/projects/:id/comments

POST /api/v1/projects/:id/comments

GET /api/v1/search

GET /api/v1/discover

GET /api/v1/projects/:id/analytics

GET /api/v1/github/repositories

POST /api/v1/github/import

GET /api/v1/notifications

Use consistent:

- HTTP status codes
- Error format
- Validation
- Pagination
- Sorting
- Filtering

---

43. API Response Format

Use consistent API responses.

Success:

```
{
  "success": true,
  "data": {},
  "message": "Project created successfully"
}
```

Error:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid project data",
    "details": []
  }
}
```

Do not expose internal stack traces in production.

---

44. Security

Implement:

- Password hashing
- JWT authentication
- Refresh-token rotation where appropriate
- Authorization middleware
- Input validation
- MongoDB injection protection
- XSS protection
- CORS configuration
- Rate limiting
- Secure cookies
- File upload validation

- Request size limits
- Helmet
- Environment variables
- Secure error handling

Never commit:

- JWT secrets
- GitHub client secrets
- AWS credentials
- Database passwords
- API keys

---

45. Performance

Implement:

- MongoDB indexes
- Pagination
- Query projections
- Efficient aggregation pipelines
- Redis caching
- Lazy loading
- Image optimization
- API response compression
- Rate limiting
- Avoid N+1-style application queries

Discovery/search APIs must not load entire project documents unnecessarily.

Load detailed project sections only when required.

---

46. Redis

Use Redis for:

- API caching
- Rate limiting
- Session-related temporary data where appropriate

- Background jobs
- Notification processing
- GitHub API response caching

Do not make Redis a required dependency for basic database persistence.

MongoDB remains the source of truth.

---

47. Background Jobs

Use **BullMQ** with Redis.

Potential jobs:

- GitHub repository synchronization
- Notification delivery
- Analytics aggregation
- Image processing
- Search index updates
- Email processing

Jobs should be retryable.

Implement logging for failed jobs.

---

48. Testing

Backend:

- Unit tests
- Controller tests
- Service tests
- API integration tests
- Authentication tests
- Authorization tests

Frontend:

- Component tests
- Form tests
- API integration tests
- Critical user-flow tests

Critical flows:

1. Registration
2. Login
3. Project creation
4. Project editing
5. Project publishing
6. Search
7. Like
8. Bookmark
9. Comment
10. Follow
11. GitHub import
12. Admin moderation

---

49. API Documentation

Use Swagger/OpenAPI.

Expose:

/api/docs

Documentation should include:

- Endpoint description
- Authentication
- Request schema
- Response schema
- Error responses
- Example requests
- Example responses

---

50. Environment Configuration

Create:

.env.example

Variables should include:

NODE_ENV

PORT

MONGODB_URI

JWT_ACCESS_SECRET

JWT_REFRESH_SECRET

REDIS_URL

AWS_REGION

AWS_ACCESS_KEY_ID

AWS_SECRET_ACCESS_KEY

AWS_S3_BUCKET

GITHUB_CLIENT_ID

GITHUB_CLIENT_SECRET

GITHUB_CALLBACK_URL

CLIENT_URL

SENTRY_DSN

Never commit .env.

---

51. Docker

Create:

Dockerfile

docker-compose.yml

Development services:

frontend

backend

mongodb

redis

Application should start locally using Docker Compose.

Example:

docker compose up --build

---

52. CI/CD

Use GitHub Actions.

On pull request:

- Install dependencies
- Run lint
- Run type checking
- Run frontend tests
- Run backend tests
- Build frontend
- Build backend

On main branch:

- Run complete test suite
- Build production images
- Push Docker images
- Prepare deployment

Do not automatically deploy to production without clearly configured secrets/environment protection.

---

53. AWS Deployment

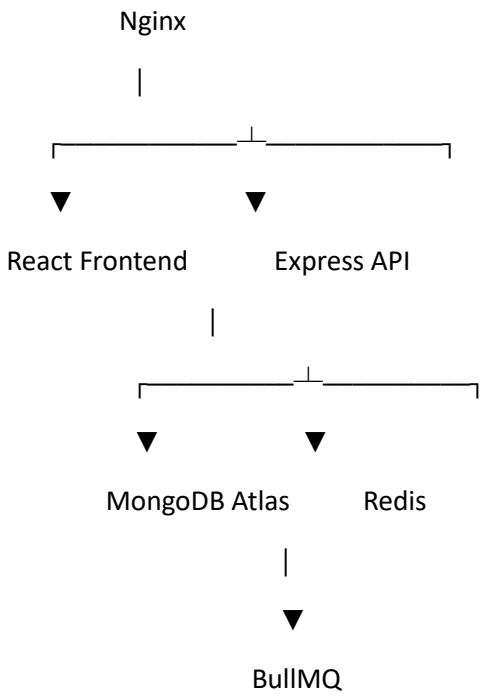
Design the application so it can be deployed to AWS.

Suggested architecture:

Cloudflare / DNS

|

▼



Possible deployment options:

Frontend:

- S3 + CloudFront
- AWS Amplify
- Vercel

Backend:

- AWS EC2
- AWS ECS

Database:

- MongoDB Atlas

Storage:

- AWS S3

CI/CD:

- GitHub Actions

The code should remain deployment-provider agnostic where reasonable.

---

54. Monitoring

Integrate Sentry.

Track:

- Backend exceptions
- Frontend exceptions
- API failures
- Failed background jobs

Do not log:

- Passwords
- JWT secrets
- API keys
- OAuth tokens
- Sensitive user data

---

55. Seed Data

Create realistic seed data.

Include:

- 5–10 users
- 10–20 projects
- Multiple technology stacks
- Architecture diagrams
- Engineering decisions
- Problems and solutions
- Timeline events
- Comments
- Likes
- Bookmarks

Example projects:

- AI Resume Analyzer

- Expense Tracker
- CRM
- Trading Platform
- AI Knowledge Agent
- E-commerce Platform
- Cybersecurity Tool
- SaaS Analytics Platform
- Developer Productivity Tool
- Real-time Collaboration Platform

Do not use lorem ipsum.

---

56. Demo Account

Create a development-only demo account.

Example:

Email:

demo@buildatlas.dev

Document the password only in development documentation.

Never ship development credentials to production.

---

57. Developer Documentation

Create:

README.md

docs/

architecture.md

database.md

api.md

deployment.md

development.md

README should explain:

- Product

- Features
- Architecture
- Tech stack
- Local setup
- Environment variables
- MongoDB setup
- Redis setup
- Docker setup
- Running tests
- API documentation
- GitHub integration
- Deployment

---

58. Repository Structure

Recommended root:

```
buildatlas/
|
├── frontend/
|
├── backend/
|
├── docs/
|
├── .github/
|   └── workflows/
|
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
└── LICENSE
```

---

59. GitHub Repository Quality

Do not commit:

.env

node_modules/

dist/

build/

coverage/

logs/

Use a proper .gitignore.

Use meaningful commit messages.

Keep configuration environment-based.

---

60. Future AI Layer

AI must NOT be required for the core application.

BuildAtlas should work completely without AI.

However, architect the system so an AI layer can be added later.

Potential future features:

AI Project Summarizer

Generate summaries from documented project sections.

AI Architecture Explainer

Explain architecture diagrams.

Semantic Project Search

Search projects by meaning rather than only keywords.

Similar Projects

Find projects with similar:

- Technology stacks
- Architecture
- Features
- Problems

AI Documentation Assistant

Help developers write:

- Architecture decisions
- Problem/solution reports
- Deployment documentation
- Lessons learned

Potential future technologies:

- OpenAI API
- Anthropic API
- Gemini API
- MongoDB Atlas Vector Search
- pgvector only if a future PostgreSQL-based search service is introduced

Do not add an LLM dependency to the MVP.

---

61. MVP Development Phases

Build the application incrementally.

Phase 1 — MERN Foundation

Implement:

- React + TypeScript
- Node.js
- Express.js
- MongoDB
- Mongoose
- Authentication
- User profiles
- Project CRUD
- Technologies
- Public project pages

Phase 2 — Engineering Knowledge

Implement:

- Architecture diagrams
- Database documentation

- API documentation
- Engineering decisions
- Problems and solutions
- Timeline
- Deployment
- Lessons learned

Phase 3 — Community

Implement:

- Likes
- Comments
- Bookmarks
- Following
- Notifications

Phase 4 — Platform

Implement:

- Search
- Filters
- Analytics
- GitHub integration
- GitHub import
- Admin dashboard
- Reporting
- SEO

Phase 5 — Production

Implement:

- Redis
- BullMQ
- S3
- Docker
- GitHub Actions
- AWS deployment

- Sentry
- Performance optimization
- Security hardening

Phase 6 — Optional AI

Implement:

- Semantic search
- AI project summaries
- AI documentation assistance
- Similar project recommendations

---

62. Definition of Done

The project is complete when:

- A user can register.
- A user can log in.
- A user can create a developer profile.
- A user can create a project.
- A user can edit a project.
- A user can document its complete engineering story.
- A user can add technologies.
- A user can create an interactive architecture diagram.
- A user can document MongoDB data models.
- A user can document APIs.
- A user can document engineering decisions.
- A user can document problems and solutions.
- A user can document deployment.
- A user can document lessons learned.
- A user can publish the project.
- Another user can discover the project.
- Users can search and filter projects.
- Users can like projects.
- Users can bookmark projects.

- Users can comment.
- Users can follow developers.
- Users receive notifications.
- Project owners can view analytics.
- GitHub repositories can be connected/imported.
- Admins can moderate the platform.
- Images/assets can be uploaded.
- The application works responsively.
- Tests cover critical functionality.
- Docker can run the development environment.
- CI/CD is configured.
- API documentation is available.
- Environment variables are properly managed.
- The application can be deployed to AWS.
- Production errors can be monitored.
- No secrets are committed to the repository.

---

63. Implementation Instructions for the CLI Coding Agent

Do not build the entire project in one step.

First analyze the requirements.

Then:

1. Design the architecture.
2. Create repository structure.
3. Initialize frontend.
4. Initialize backend.
5. Configure TypeScript.
6. Configure MongoDB/Mongoose.
7. Define database models.
8. Define API contracts.
9. Implement authentication.
10. Implement project CRUD.

11. Implement frontend project management.
12. Implement public project pages.
13. Implement engineering documentation modules.
14. Implement community features.
15. Implement search.
16. Implement GitHub integration.
17. Implement analytics.
18. Implement Redis/BullMQ.
19. Implement S3.
20. Add Docker.
21. Add tests.
22. Add CI/CD.
23. Prepare AWS deployment.

After each major phase:

- Run tests.
- Run linting.
- Run TypeScript checks.
- Start the application.
- Verify APIs.
- Verify frontend/backend integration.
- Fix errors before moving forward.

Do not leave core functionality as placeholders.

Do not use fake API responses once the backend is implemented.

Do not hardcode project data in React.

Do not use mock data as the permanent data source.

MongoDB must be the actual persistence layer.

Use proper Mongoose models.

Use migrations/versioned database initialization where appropriate.

Use reusable React components.

Use proper loading states.

Use proper error states.

Use empty states.

Use form validation.

Use pagination.

Use authentication and authorization.

Prioritize engineering quality over visual effects.

---

64. Final Product Goal

The final product should allow a developer to visit a project and understand:

What was built?

Why was it built?

How was it architected?

Why were these technologies selected?

How does the MongoDB data model work?

How do the APIs work?

What engineering decisions were made?

What problems occurred?

How were those problems solved?

How was the application deployed?

What did the developer learn?

What would they change next time?

The final product should feel like a **professional MERN-based engineering knowledge and project discovery platform**, not a student CRUD application.

The core identity of BuildAtlas is:

GitHub shows the code. BuildAtlas shows how the software was built.